

238. Product of Array Except Self Medium

Acceptance: 68.5% | [LeetCode](#)

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

```
**Input:** nums = [1,2,3,4]
**Output:** [24,12,8,6]
```

Example 2:

```
**Input:** nums = [-1,1,0,-3,3]
**Output:** [0,0,9,0,0]
```

Constraints:

- $2 \leq \text{nums.length} \leq 105$
- $-30 \leq \text{nums}[i] \leq 30$
- The input is generated such that `answer[i]` is **guaranteed** to fit in a **32-bit** integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array **does not** count as extra space for space complexity analysis.)

Claude Opus Solution

Product of Array Except Self

1. Intuition

The **key insight** is recognizing what "product of all elements except self" actually means:

```
answer[i] = (product of all elements to the LEFT of i) × (product of all elements to the RIGHT of i)
```

For example, with `nums = [1, 2, 3, 4]`:

```
answer[2] = (1 × 2) × (4) = 2 × 4 = 8
           └left┘   └right┘
```

Since we **can't use division**, we can't simply compute the total product and divide by `nums[i]` (which would also break for zeros). Instead, we compute the **prefix products** (left side) and **suffix products** (right side) separately, then multiply them.

This is a classic **prefix/suffix accumulation** pattern.

2. Approach

- Create output array, initialize all to 1.
- First pass (left → right): build prefix products
 - Maintain running 'prefix' = product of everything before index `i`
 - Store prefix in `output[i]`
 - Then update `prefix *= nums[i]`
- Second pass (right → left): build suffix products
 - Maintain running 'suffix' = product of everything after index `i`
 - Multiply `output[i] *= suffix`
 - Then update `suffix *= nums[i]`
- Return output array.

The trick to achieving **$O(1)$ extra space** is reusing the output array to first store prefix products, then multiplying in suffix products on the fly (so we never need a separate suffix array).

3. Python Solution

```
from typing import List

class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
```

```

n = len(nums)

# output[i] will eventually hold the product of all elements except nums[i].
# We initialize to 1 because we'll be multiplying into these slots.
output = [1] * n

# ---- PASS 1: Prefix products (left side) ----
# 'prefix' accumulates the product of all elements BEFORE index i.
# For the very first element, there are no elements to the left, so prefix = 1.
prefix = 1
for i in range(n):
    output[i] = prefix      # store product of everything to the left of i
    prefix *= nums[i]      # extend prefix to include nums[i] for the next index

# After pass 1 for [1,2,3,4], output = [1, 1, 2, 6]
# output[0] = 1      (nothing to left)
# output[1] = 1      (= nums[0])
# output[2] = 1*2 = 2 (= nums[0]*nums[1])
# output[3] = 1*2*3=6 (= nums[0]*nums[1]*nums[2])

# ---- PASS 2: Suffix products (right side) ----
# 'suffix' accumulates the product of all elements AFTER index i.
# For the last element, there are no elements to the right, so suffix = 1.
suffix = 1
for i in range(n - 1, -1, -1):
    output[i] *= suffix    # combine left product with right product
    suffix *= nums[i]     # extend suffix to include nums[i] for next index

# After pass 2 for [1,2,3,4]:
# output[3] = 6 * 1 = 6
# output[2] = 2 * 4 = 8
# output[1] = 1 * 12 = 12 (suffix = 4*3)
# output[0] = 1 * 24 = 24 (suffix = 4*3*2)
# Final: [24, 12, 8, 6] ✓

return output

```

Walking through Example 2 (with a zero): `nums = [-1, 1, 0, -3, 3]`

```

Pass 1 (prefix):
output = [1, -1, -1, 0, 0]
(once we pass the 0, all later prefixes become 0)

Pass 2 (suffix):
i=4: output[4] = 0 * 1 = 0;    suffix = 3
i=3: output[3] = 0 * 3 = 0;    suffix = 3*-3 = -9
i=2: output[2] = -1 * -9 = 9;  suffix = -9*0 = 0
i=1: output[1] = -1 * 0 = 0;   suffix = 0
i=0: output[0] = 1 * 0 = 0;    suffix = 0

Final: [0, 0, 9, 0, 0] ✓

```

Notice how the prefix/suffix approach naturally handles zeros — something division could never do cleanly.

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(n)$	Two separate passes over the array, each $O(n)$. Total $O(2n) = O(n)$.
Space	$O(1)$	We only use two scalar variables (<code>prefix</code> , <code>suffix</code>). The output array does not count as extra space per the problem statement.

This satisfies **both** constraints: linear time **and** the follow-up $O(1)$ extra space.

5. Alternative Approaches

A) Two Auxiliary Arrays (more intuitive, less optimal space)

Build separate `left[]` and `right[]` prefix/suffix arrays, then `answer[i] = left[i] * right[i]`. This is easier to understand but uses **$O(n)$ extra space**. (This is essentially `my_solution.py` from the reference.)

```

left[i] = product of nums[0..i-1]
right[i] = product of nums[i+1..n-1]
answer[i] = left[i] * right[i]

```

B) Single-Loop Two-Pointer (most compact)

Combine both passes into **one loop** by simultaneously updating from the front and back (this is `solution2.py`):

```

output = [1] * len(nums)
prefix = suffix = 1
for i in range(len(nums)):
    output[i] *= prefix
    prefix *= nums[i]
    output[-1 - i] *= suffix    # work from the back too
    suffix *= nums[-1 - i]
return output

```

Same $O(n)$ time and $O(1)$ space, just fewer lines — though arguably less readable for beginners.

C) Division Approach (❌ disallowed)

Compute total product, then `answer[i] = total / nums[i]`. This fails because: - The problem **forbids division**. - It breaks when any element is **zero** (division by zero / incorrect results).

Recommended: The two-pass solution above strikes the best balance between clarity and optimality. Master this **prefix/suffix pattern** — it appears in many problems (e.g., "Trapping Rain Water", "Candy", running min/max problems).

Codex Solution

1. Intuition

For each index `i`, we need:

```

answer[i] = product of everything before i * product of everything after i

```

Example:

```

nums = [1, 2, 3, 4]

answer[2] = nums[0] * nums[1] * nums[3]
           = prefix product before index 2 * suffix product after index 2
           = 1 * 2 * 4
           = 8

```

So the key pattern is:

Build the answer using prefix products from the left and suffix products from the right.

We cannot use division, so instead of computing total product and dividing by `nums[i]`, we compute the left-side and right-side products directly.

2. Approach

We can do this in $O(n)$ time and $O(1)$ extra space, excluding the output array.

Step-by-step

Create answer array initialized with 1s

```

prefix = 1

```

Move from left to right:

```

answer[i] = prefix
prefix *= nums[i]

```

At this point:

`answer[i]` contains product of all elements to the left of `i`

```

suffix = 1

```

Move from right to left:

```
answer[i] *= suffix
suffix *= nums[i]
```

At this point:

```
answer[i] = left product * right product
```

Return answer

3. Python Solution

```
from typing import List
```

```
class Solution:
```

```
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
```

```
        # answer[i] will eventually contain:
        # product of all nums[j] where j != i
        #
        # We initialize with 1 because 1 is the multiplication identity.
        answer = [1] * n
```

```
        # -----
        # First pass: compute prefix products
        # -----
        # prefix stores the product of all elements to the LEFT of i.
        #
        # Example:
        # nums = [1, 2, 3, 4]
        #
        # At i = 0, prefix = 1           -> answer[0] = 1
        # At i = 1, prefix = 1           -> answer[1] = 1
        # At i = 2, prefix = 1 * 2 = 2   -> answer[2] = 2
        # At i = 3, prefix = 1 * 2 * 3 = 6 -> answer[3] = 6
        #
        # After this pass:
        # answer = [1, 1, 2, 6]
        prefix = 1
```

```
        for i in range(n):
            # Store product of everything before index i
            answer[i] = prefix

            # Update prefix to include nums[i] for the next index
            prefix *= nums[i]
```

```
        # -----
        # Second pass: compute suffix products and multiply them in
        # -----
        # suffix stores the product of all elements to the RIGHT of i.
        #
        # Continuing the example:
        # nums = [1, 2, 3, 4]
        # answer currently = [1, 1, 2, 6]
        #
        # At i = 3, suffix = 1           -> answer[3] = 6 * 1 = 6
        # At i = 2, suffix = 4           -> answer[2] = 2 * 4 = 8
        # At i = 1, suffix = 12          -> answer[1] = 1 * 12 = 12
        # At i = 0, suffix = 24          -> answer[0] = 1 * 24 = 24
        #
        # Final answer = [24, 12, 8, 6]
        suffix = 1
```

```
        for i in range(n - 1, -1, -1):
            # Multiply current left-product by product of everything after i
            answer[i] *= suffix

            # Update suffix to include nums[i] for the next index to the left
```

```
    suffix *= nums[i]

    return answer
```

4. Complexity Analysis

Time Complexity

$O(n)$

We make two passes through the array:

1. One left-to-right pass for prefix products
2. One right-to-left pass for suffix products

So the total work is:

$O(n) + O(n) = O(n)$

Space Complexity

$O(1)$

We only use a few extra variables:

```
prefix
suffix
n
```

The `answer` array is required as the output and does not count as extra space according to the problem statement.

5. Alternative Approaches

Alternative 1: Use separate prefix and suffix arrays

You can create two arrays:

```
prefix[i] = product of everything before i
suffix[i] = product of everything after i
```

Then:

```
answer[i] = prefix[i] * suffix[i]
```

This is easy to understand but uses $O(n)$ extra space.

Alternative 2: Use division

If division were allowed, we could compute the total product and divide by `nums[i]`.

However, this problem explicitly disallows division. Also, division becomes tricky when the array contains zeroes.

For example:

```
nums = [-1, 1, 0, -3, 3]
```

The prefix/suffix approach handles zeroes naturally without any special cases.

Key Pattern

This is a classic **prefix/suffix product** problem.

Use this pattern when:

- You need information from all elements except the current one
- The operation can be accumulated from left and right
- Division or inverse operations are unavailable or unsafe